

C++ Implementations of Lie Group Variational Integrators and Nonlinear Geometric PD Control of a Spacecraft

Yoonjae Hwang

Abstract—For this project, I studied geometric structure-preserving numerical methods for ODEs, and geometric control. I started from the source code provided from the 3F2 extension pack. The Eigen library was used to construct non-trivial matrix operations. The main papers that I used to study this were Leok et al's work in geometric integration, and T.Lee's work in geometric control of UAVs.

I. GEOMETRIC INTEGRATION

Traditional numerical integration methods such as the Runge-Kutta or Verlet schemes are limited in their accuracy, as they do not inherently include the conservation principles. Physical systems, under certain circumstances have quantities that are conserved, such as angular/linear momentum, symplectic form, and other geometric structures that exist in the configuration space. Geometric integrators are developed to preserve these structures, allowing for long time accuracy in its qualitative behaviour.

A. Lie Group Variational Integrators

The attitude of a rigid body can be described by a rotation matrix $R \in SO(3) = \{R \in \mathbb{R}^{3 \times 3} \mid R^T R = I, \det(R) = 1\}$. The set of rotation matrices $SO(3)$, is an example of a Lie Group. The attitude and position of a rigid body can be described by the Lie Group $SE(3)$. The discrete equations of motion used for the numerical integration schemes are derived by discretising the Lagrangian, and then taking a discrete variational approach to Hamilton's Principle to find the equations of motion.

B. Continuous Time Equations of Motion

For our spacecraft, we can model the system by the following continuous time equations of motion.

$$\begin{aligned} \dot{x} &= v \\ m\dot{v} &= -\nabla U(x) + F_T + F_D \\ \dot{R} &= RS(\Omega) \\ \dot{\Pi} + \Omega \times \Pi &= M + \tau_T + \tau_D \end{aligned} \quad (1)$$

$U(x)$ is the potential energy function around Mars, F_T and F_D are forces from thrust and drag, and τ_T and τ_D are corresponding torque quantities. M represents the gravity gradient torque. R is the rotation matrix from body-fixed frame to the inertial frame of reference, which has its origin at the center of Mars. $S(\cdot)$ mapping produces the skew symmetric matrix upon acting on a vector, as the following:

$$S(x) = \begin{bmatrix} 0 & -x_3 & x_2 \\ x_3 & 0 & -x_1 \\ -x_2 & x_1 & 0 \end{bmatrix}. \quad (2)$$

Π represents the angular momentum, and Ω represents the angular velocity, both in the body fixed frame. The force produced from the fuel ejecting and mass decreasing is not modelled.

C. Discrete Equations of Motion

The discrete equations of motion in the Hamiltonian form:

$$\begin{aligned} x_{k+1} &= x_k + \frac{h}{m} \gamma_k + \frac{h^2}{2m} (-\nabla U(x_k) + f(x_k, p_k)) \\ p_{k+1} &= p_k + \frac{h}{2} (-\nabla U(x_k) + f(x_k, p_k)) \\ &\quad + \frac{h}{2} (-\nabla U(x_{k+1}) + f(x_{k+1}, p_{k+1})) \\ hS(\Pi_k + \frac{h}{2}(M_k + \tau_k)) &= F_k J_d + J_d F_k^T \\ \Pi_{k+1} &= F_k^T \Pi_k + \frac{h}{2} F_k^T (M_k + \tau_k) + \frac{h}{2} (M_{k+1} + \tau_{k+1}) \\ R_{k+1} &= R_k F_k \end{aligned} \quad (3)$$

Equation (3a) can be solved by considering the Lie Algebra of F_k , and using a newton iteration solver. In the various papers by M Leok, it is explained why F_k always remains on $SO(3)$.

II. GEOMETRIC ATTITUDE CONTROL

The control inputs, are the 4 throttle inputs for each of the engines. The torque produced by the thrusters can be found as shown in the extension pack. Thus, we consider the desired moment that needs to be produced by the engines, as we can simply invert the matrix that relates the torque to the throttle values. As shown in the paper by T.Lee et al (Geometric tracking control of a quadrotor UAV on $SE(3)$), we can define the error in attitude not by naively subtracting the euler angles, but rather intrinsically on $SO(3)$.

The error in attitude becomes:

$$e_R = \frac{1}{2} (R_d^T R - R^T R_d)^V \quad (4)$$

The $(\cdot)^V$ map is the inverse of the $S(\cdot)$ map. The error in angular velocity then is defined to be:

$$e_\Omega = \Omega - R^T R_d \Omega_d \quad (5)$$

Then the control input, or desired torque is:

$$\begin{aligned} \tau_u &= -k_R e_R - k_\Omega e_\Omega + \Omega \times J\Omega \\ &\quad + -J(S(\Omega)R^T R_d \Omega_d - R^T R_d \dot{\Omega}_d) \end{aligned} \quad (6)$$

III. LIMITATION

Due to the time constraints, I have not been able to run and test the implementation of the integration of the integrators/controllers within the pre-existing Mars Lander Framework. Some difficulties were working around most of the code being written in vector, instead of Eigen.

IV. COMPILATION

Please compile the `LGVI_SE3.cpp` C++ file as well as the two files provided from the department. An heading with for the LGVI file is provided by `LGVI_SE3.h`.